Musterlösung zur Klausur zu OP

1. Aufgabe: Versionierung / Versionsverwaltung (5 Punkte)

Erläutern Sie kurz (2-3 Sätze): Was versteht man im Zusammenhang mit Versionsverwaltung unter *Einchecken mit einem Konflikt?*

Basisanwendung

Für die folgenden Aufgaben ist eine Anwendung zur Verwaltung von Sportvereinen als Basis gegeben, siehe die folgenden Seiten.

Klasse Sportverein

```
public class Sportverein {

  private String name;
  private int anzahlMitglieder;
  private int anzahlAktiveMitglieder;

  // Standard-Konstruktor, welcher die Attribute mit Werten seiner
  // Parameter belegt und get- und set-Methoden zu den Attributen

  @Override
  public String toString() {
        String ergebnis = this.getName() + "\n"
              + this.getAnzahlMitglieder() + "\n"
              + this.getAnzahlAktiveMitglieder();
        return ergebnis;
  }
}
```

Klasse Main

```
public class Main {

  public static void main(String[] args) {
        SportvereineControl sportvereineControl
              = new SportvereineControl();
        sportvereineControl.fuelleSportvereine();
        sportvereineControl.zeigeSportvereineAn();
  }
}
```

Klasse SportvereineModel

```
public class SportvereineModel {

   private Sportverein[] sportvereine = new Sportverein[2];

   public void fuelleSportvereine() {
         this.sportvereine[0]
               = new Sportverein("Vfl Bochum", 24162, 19456);
         this.sportvereine[1]
               = new Sportverein("FC Schalke 04", 173523, 150432);
   }

   public String gibSportvereineAus() {
         String ergebnis = "";
         for(Sportverein v : this.sportvereine) {
               ergebnis = ergebnis + v.toString() + "\n\n";
         }
         return ergebnis;
   }
}
```

Klasse SportvereineControl

```
public class SportvereineControl {

   private SportvereineModel sportvereineModel;
   private SportvereineView sportvereineView;

   public SportvereineControl() {
         this.sportvereineModel = new SportvereineModel();
         this.sportvereineView
               = new  SportvereineView(this.sportvereineModel, this);
   }

   public void fuelleSportvereine() {
         this.sportvereineModel.fuelleSportvereine();
   }

   public void zeigeSportvereineAn() {
         this.sportvereineView.zeigeSportvereineAn();
   }
}
```

Klasse SportvereineView

```
import javax.swing.JOptionPane;

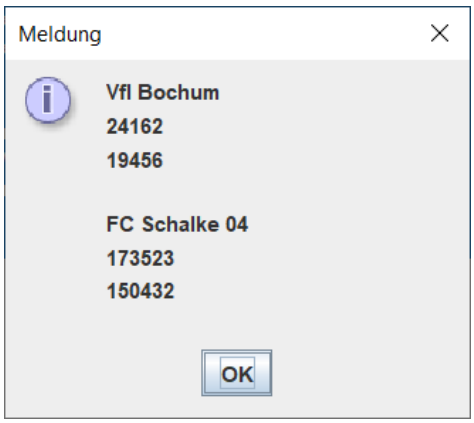
public class SportvereineView {

   SportvereineModel sportvereineModel;
   SportvereineControl sportvereineControl;

   public SportvereineView(SportvereineModel sportvereineModel,
         SportvereineControl sportvereineControl) {
         this.sportvereineModel = sportvereineModel;
         this.sportvereineControl = sportvereineControl;
   }

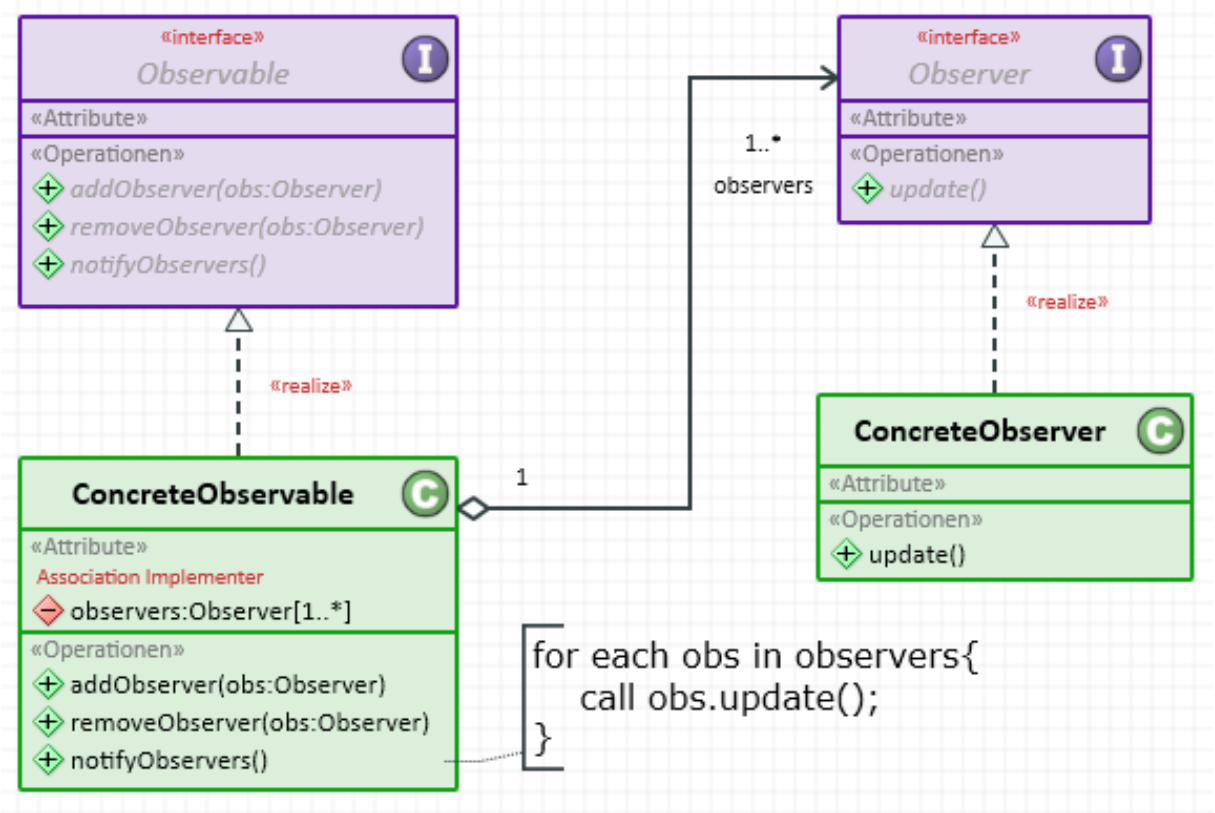
   public void zeigeSportvereineAn() {
         JOptionPane.showMessageDialog(null,
               this.sportvereineModel.gibSportvereineAus());
   }
}
```

Startet man die Anwendung, so erhält man das folgende Ausgabefenster.



2. Aufgabe: Design Patterns (40 Punkte)

Gegeben ist die Basisanwendung, die Anwendung zur Verwaltung von Sportvereinen. Diese Aufgabe führen Sie auf der Basis dieser Anwendung aus.
Gegeben ist weiterhin ein UML-Klassendiagramm zum Entwurfsmuster **Observer.**



Karl Eilebrecht, Gernot Starke, Patterns kompakt

Geben Sie den Quellcode an, der aus dem UML-Klassendiagramm zu dem Pattern *Observer* hervorgeht bzw. ergänzen Sie die vorliegenden Klassen der Basisanwendung, so dass diese das Observer Pattern umsetzt:

Welche Klasse wird zum *ConcreteObserver*, welche zum ConcreteObservable?

Übernehmen Sie also nur die Namen der Interfaces *Observer, Observable* nicht die der Klassen *ConcreteObserver und ConcreteObservable* .

Geben Sie den Quellcode der neuen Methoden aus Ihrer *ConcreteObserver*-Klasse und Ihrer *ConcreteObservable*-Klasse an. Die bisherige Funktionalität soll sich insofern ändern, dass immer dann, wenn die Methode *fuelleSportvereine* der Klasse *SportvereineModel* aufgerufen wird, automatisch die Anzeige in der Konsole erfolgt. Aus der Klasse *Main* entfällt daher in der *main*-Methode die Zeile *sportvereineControl.zeigeSportvereineAn()*.

Lösung

Interface Observable

```
public interface Observable {

    public void addObserver(Observer obs);
    public void removeObserver(Observer obs);
    public void notifyObservers();
}
```

Interface Observer

```
 public interface Observer {

    public void update();
}
```

Klasse SportvereineModel

```
import java.util.Vector;

public class SportvereineModel implements Observable {

    private Sportverein[] sportvereine = new Sportverein[2];
    private Vector <Observer> observers
          = new Vector <Observer>();
    public void fuelleSportvereine() {
          this.sportvereine[0]
                = new Sportverein("Vfl Bochum", 24162, 19456);
          this.sportvereine[1]
                = new Sportverein("FC Schalke 04", 173523, 150432);
          notifyObservers();
    }

    public String gibSportvereineAus() {
          String ergebnis = "";
          for(Sportverein v : this.sportvereine) {
                ergebnis = ergebnis + v.toString() + "\n\n";
          }
          return ergebnis;
    }

    public void addObserver(Observer obs) {
          this.observers.add(obs);
    }

    public void removeObserver(Observer obs) {
          this.observers.remove(obs);
    }
```

```
    public void notifyObservers() {
          for (Observer o : this.observers) {
                o.update();
          }
    }
}
```

Klasse SportvereineControl

```
public class SportvereineControl implements Observer{

    private SportvereineModel sportvereineModel;
    private SportvereineView sportvereineView;

    public SportvereineControl() {
          this.sportvereineModel = new SportvereineModel();
          this.sportvereineModel.addObserver(this);
          this.sportvereineView
                = new SportvereineView(this.sportvereineModel, this);
    }

    public void fuelleSportvereine() {
          this.sportvereineModel.fuelleSportvereine();
    }

    public void zeigeSportvereineAn(){
    @Override
    public void update() {
          this.sportvereineView.zeigeSportvereineAn();
    }

}
```

Klasse SportvereineView

Keine Änderungen!

ACHTUNG: Es ist auch möglich, SportvereineView zum Observer zu machen!

3. Aufgabe: Vertiefung Java (30 Punkte)

Gegeben ist die Basisanwendung zur Verwaltung von Sportvereinen und eine weitere Klasse *NachhaltigerSportverein*. Packages und Importe werden nicht berücksichtigt.

```
public class NachhaltigerSportverein extends Sportverein{

  private boolean setztSolarthermieEin;
  private boolean setztPhotovoltaikEin;

  // Konstruktor mit Parametern und get- und is-Methoden

  @Override
  public String toString() {
        String ergebnis = super.toString() + "\n"
              + this.isSetztSolarthermieEin() + "\n"
              + this.isSetztPhotovoltaikEin();
        return ergebnis;
  }
}
```

a) Generics, for each

Geben Sie den Quellcode an, um in der Klasse *SportvereineModel* das Attribut *sportvereine* vom Typ *Sportverein[ ]* zu ersetzen durch ein Objekt vom Typ *ArrayList* unter Verwendung von Generics. Geben Sie weiterhin den neuen Quellcode der Methode *fuelleSportvereine* der Klasse *SportvereineModel* an.

b) Generics mit Typeinschränkungen

Geben Sie den Quellcode einer Klasse *SportvereinKonsole* an, welche ein Generic *T* enthält, für welches nur ein Objekt vom Typ *Sportverein* oder einer Ableitung von *Sportverein* eingesetzt werden kann. Es gibt ein privates Attribut *sportvereinBasis* vom Typ *T*, einen Konstruktor, welcher das Attribut mit einem Wert belegt, und eine Methode *gibAttributeInKonsoleAus* ohne Parameter und Rückgabewert. Diese schreibt die Attribute von *sportvereinBasis* in die Konsole, siehe unten. (*SportvereinKonsole* soll beispielsweise für Testausgaben genutzt werden.)

Erstellen Sie in der Klasse *SportvereineControl* folgendermaßen eine Methode *gibSportvereineInKonsoleAus.* Diese hat keine Parameter und keine Rückgabewerte. Erstellen Sie innerhalb dieser Methode ein Objekt *sp1* vom Typ *Sportverein* mit den Werten *Mustersportverein1, 654321, 500000* und ein Objekt *sp2* vom Typ *NachhaltigerSportverein* mit den Werten *Mustersportverein2, 123456, 120000, true, true* und zwei Objekte *sp1Konsole* und *sp2Konsole* vom Typ *SportvereinKonsole* unter Verwendung der Objekte *sp1* und *sp2*. Beim Aufruf dieser Methode (z.B. mittels *sportvereineControl.gibSportvereineInKonsoleAus();* in der *main-Methode* der Klasse *Main*) erscheint die folgende Konsolenausgabe.

```
Die Werte der Attribute von Mustersportverein1 sind die folgenden:
Mustersportverein1
654321
500000
Die Werte der Attribute von Mustersportverein2 sind die folgenden:
Mustersportverein2
123456
120000
true
true
```

Lösung

zu a)

In Klasse SportvereineModel

```
private ArrayList<Sportverein> sportvereine = new ArrayList<>();

public void fuelleSportvereine(){
    this.sportvereine.add(
        new Sportverein("Vfl Bochum", 24162, 19456));
    this.sportvereine.add(
        new Sportverein("FC Schalke 04", 173523, 150432));
}
```

zu b)

Klasse SportvereinKonsole

```
public class SportvereinKonsole<T extends Sportverein> {

 private T sportvereinBasis;

 public SportvereinKonsole(T sportvereinBasis) {
    super();
    this.sportvereinBasis = sportvereinBasis;
 }

 public void gibAttributeInKonsoleAus() {
    System.out.println("Die Werte der Attribute von "
        + sportvereinBasis.getName() + " sind die folgenden: \n"
        + sportvereinBasis.toString());
 }
}
```

In Klasse SportvereineControl

```
public void gibSportvereineInKonsoleAus() {
  Sportverein sp1 = new Sportverein(
        "Mustersportverein1", 654321, 500000);
  NachhaltigerSportverein sp2 = new NachhaltigerSportverein(
        "Mustersportverein2", 123456, 120000, true, true);
  SportvereinKonsole<Sportverein> sp1Konsole
        = new SportvereinKonsole<Sportverein>(sp1);
  SportvereinKonsole<NachhaltigerSportverein> sp2Konsole
      = new SportvereinKonsole<NachhaltigerSportverein>(sp2);
  sp1Konsole.gibAttributeInKonsoleAus();
  sp2Konsole.gibAttributeInKonsoleAus();
}
```

4. Aufgabe: JUnit 5 und Zustandsbasierter Test (25 Punkte)

Gegeben ist die Basisanwendung zur Verwaltung von Sportvereinen.

JUnit 5 15 Punkte

Ergänzen Sie den vorliegenden Quellcode eines JUnit Tests. Der Konstruktor der Klasse *Sportverein* soll durch diese JUnit Testklasse daraufhin getestet werden, ob die Anzahl der Mitglieder und die Anzahl der aktiven Mitglieder richtig gesetzt werden. Das soll mit einem Testfall innerhalb einer Testmethode, die *Supplier<String>* verwendet, getestet werden. Weiterhin soll der Konstruktor daraufhin getestet werden, ob bei der Eingabe einer Anzahl aktiver Mitglieder, die größer ist als die Anzahl der Mitglieder eine *IllegalArgumentException* mit der Meldung *Die Anzahl der aktiven Mitglieder darf nicht größer sein als die Anzahl der Mitglieder.* geworfen wird. Benutzen Sie für das Testen aus der Klasse *Assertions*
assertTrue(Boolean condition, Supplier<String> messageSupplier) und
assertEquals(String actual, String expected) und
<T> assertThrows(Class<T> clazz, Executable executable).

```
import static org.junit.jupiter.api.Assertions.*;
import  org.junit.jupiter.api.*;
public class SportvereinTest{

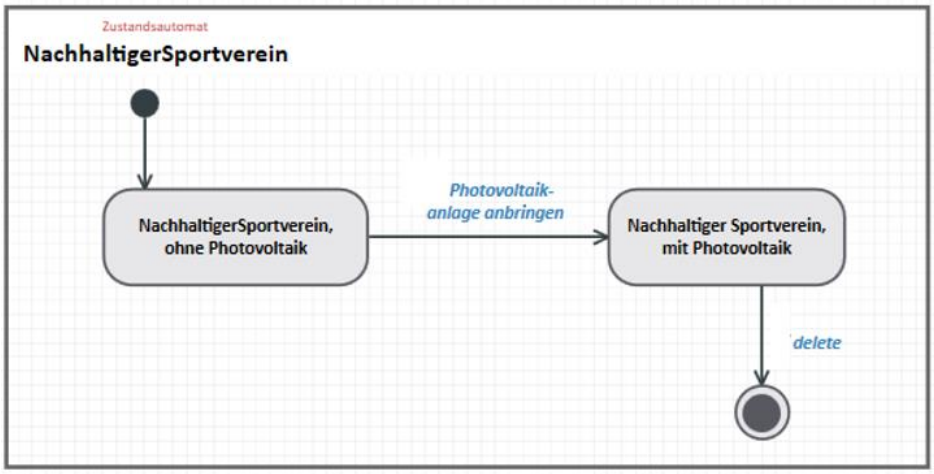
  @Test
  public void testSportverein() throws Exception{
        // Ergaenzen!!!
        Sportverein sp = new Sportverein("Mustersportverein1",
              333, 222);
        assertTrue(sp.getAnzahlMitglieder() == 333
              && sp.getAnzahlAktiveMitglieder() == 222,
              ()-> "Die Anzahl der Mitglieder und der aktiven "
              + "Mitglieder wird nicht korrekt gesetzt.");

  }

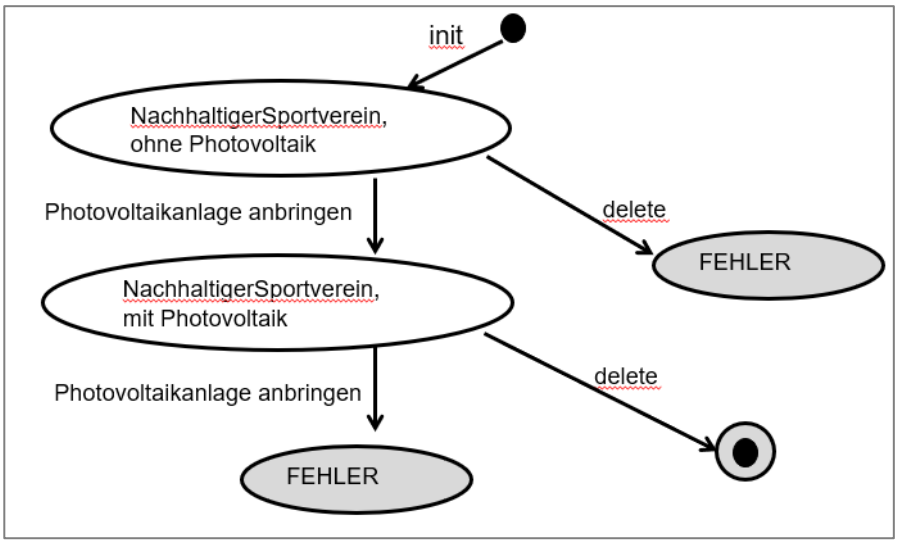
  @Test
  public void testSportvereinException() throws Exception{
        // Ergaenzen!!!
        Throwable exc = assertThrows(IllegalArgumentException.class,
              () -> {
                    new Sportverein("Mustersportverein2", 5, 55);
              });
        assertEquals("Die Anzahl der aktiven Mitglieder darf "
              + " nicht größer sein als die Anzahl der Mitglieder.",
              exc.getMessage());
  }
}
```

Zustandsbasierter Test 10 Punkte

Gegeben ist ein UML Zustandsautomat, welcher die Zustände eines Objekts vom Typ *NachhaltigerSportverein* der Aufgabe 3 bezüglich der Eigenschaft *Photovoltaik* modelliert. Geben Sie den Übergangsbaum an.
Geben Sie zwei Testfälle inklusive Sollergebnis an, die sich aus dem Übergangsbaum ergeben.



Lösung



Testfälle:

init / Photovoltaikanlage anbringen / delete -> Ein NachhaltigerSportverein-Objekt, ohne Photovoltaik, wurde angelegt, eine Photovoltaikanlage wurde angebracht und damit das Attribut setztPhotovoltaik ein auf true gesetzt, und dann gelöscht.

init / delete -> Ein NachhaltigerSportverein-Objekt, ohne Photovoltaik, wurde angelegt und dann gelöscht. Fehlermeldung beim Löschversuch.

5. Aufgabe: SOLID und Clean Code (20 Punkte)

Gegeben ist die Anwendung zur Verwaltung von Sportvereinen inklusive der Klasse *NachhaltigerSportverein* aus der 3. Aufgabe.

a) 6 Punkte

Erläutern Sie das *Liskov Substitution Principle.* Nennen Sie ein Beispiel unter Verwendung der Klassen *Sportverein* und *NachhaltigerSportverein*.

Unterklassen sollen anstelle ihrer Oberklassen einsetzbar sein. Eine Instanz einer Unterklasse muss sich so verhalten, dass jemand, der meint, ein Objekt der Oberklasse vor sich zu haben, nicht durch unerwartetes Verhalten überrascht wird, wenn es sich dabei tatsächlich um ein Objekt der Unterklasse handelt.

Beispiel: Die Methode toString darf sich im Wesentlichen nicht unter- unterschiedlich verhalten bezüglich *Sportverein* und *Nachhaltiger-Sportverein*..

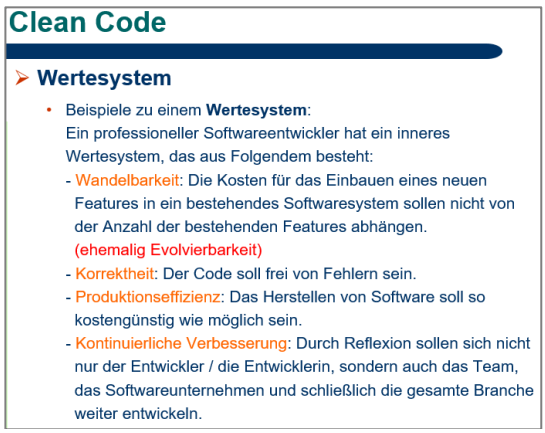
b) 4 Punkte

Wieso trägt das *Liskov Substitution Principle* zu einer höheren Wartbarkeit und Lebensdauer von Software und zu einer Reduktion der Fehleranfälligkeit bei (2-3 Sätze)?

Durch das Prinzip wird garantiert, dass Operationen vom Typ *Oberklasse*, die auf ein Objekt des Typs *Unterklasse* angewendet werden, auch korrekt ausgeführt werden.

c) 4 Punkte

Nennen und erläutern Sie kurz zwei der 4 Werte aus Clean Code.



d) 6 Punkte

Erläutern Sie kurz das Prinzip *Favour Composition over Inheritance (FCoI),* welches dem roten Grad aus Clean Code zugeordnet ist. Was ist mit *Composition* gemeint, was mit *Inheritance*? Ist das Prinzip bereits in der Verwaltung zu Sportvereinen umgesetzt worden? Begründen Sie kurz Ihre Antwort.

Favour Composition over Inheritance (FCoI) bedeutet, dass man lieber Kompositionen benutzen sollte, als vererben sollte. Das Prinzip wurde in dem Beispiel nicht umgesetzt, da es bisher keine Komposition vorhanden ist, aber eine Vererbung.